国立大学法人
電気通信大学
The University of Electro-Communications

Pham Laboratory
Integrated circuit design laboratory

「 Course 」
[Course name]

「 Lesson 4 」 Post-quantum EDHOC

[Author]

[YYYY/MM]

Today’s lesson purpose:

- See why today's EDHOC suites will not survive a quantum computer, and what ML-KEM (FIPS 203) offers instead.
- Understand EDHOC-AuthKEM: five messages that authenticate both sides with key encapsulation only — no signatures, no Diffie-Hellman, no pre-shared key.
- Build and run the post-quantum suites TBD1 / TBD2 / TBD3 (TBD3 = AES-256 + ML-KEM-1024) on the boards, inside 64 KiB.

Code: fpga/sw/export/edhoc-authkem · drafts: draft-ietf-lake-authkem-edhoc-01, draft-ietf-lake-pqsuites-01, draft-ietf-jose-pqc-kem-06

Outline

1. Why Post-Quantum
2. The AuthKEM Protocol
3. Post-Quantum Suites
4. Fitting in 64 KiB
5. What to Expect
6. Practice

Outline

1. **Why Post-Quantum**
2. The AuthKEM Protocol
3. Post-Quantum Suites
4. Fitting in 64 KiB
5. What to Expect
6. Practice

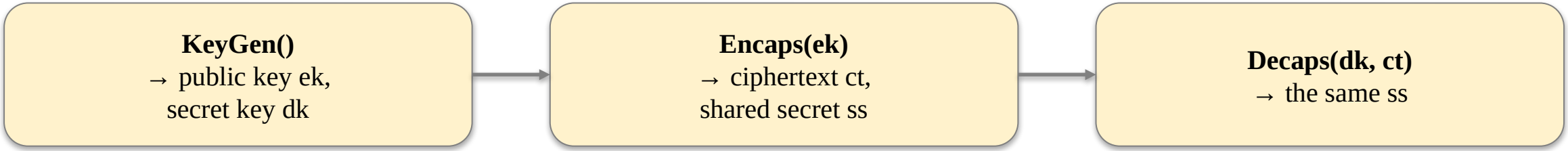
1. Why Post-Quantum (1/8) What a quantum computer breaks

Building block	Used in EDHOC suites 0–25 for	Large quantum computer
Elliptic-curve DH (X25519, P-256, P-384, X448)	key exchange	**broken** (Shor)
Elliptic-curve signatures (EdDSA, ECDSA)	authentication	**broken** (Shor)
AES-128 / AES-256	encryption	weakened: ~64 / ~128-bit strength (Grover)
SHA-256 / SHA-384, SHAKE256	hashing, key derivation	still fine

- **Harvest now, decrypt later**: radio traffic recorded today can be decrypted once such a machine exists — the key exchange must change first.
- Devices built today may run for 10–20 years.
- Fix: replace the elliptic-curve parts by post-quantum ones, keep AES-256 and SHA-384 / SHAKE256.

1. Why Post-Quantum (2/8) ML-KEM, the post-quantum key exchange (FIPS 203)

Parameter set	Security category	Public key	Secret key	Ciphertext	Shared secret
ML-KEM-512	1 (≈ AES-128)	800 B	1,632 B	768 B	32 B
ML-KEM-768	3 (≈ AES-192)	1,184 B	2,400 B	1,088 B	32 B
ML-KEM-1024	5 (≈ AES-256)	1,568 B	3,168 B	1,568 B	32 B



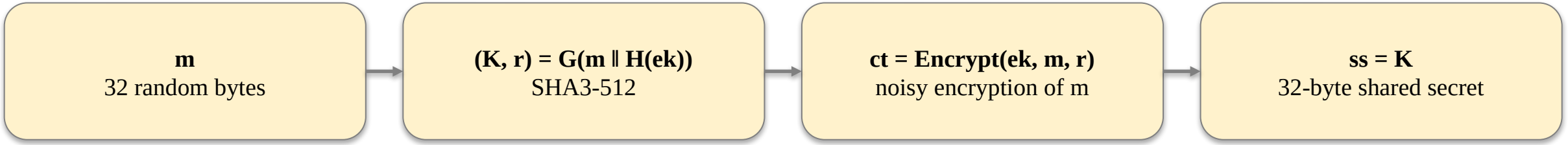
- A KEM is not Diffie-Hellman: only the holder of *dk* can recover *ss* from *ct*. That property is what AuthKEM uses for authentication.
- Keys and ciphertexts are ~10–50× larger than elliptic-curve keys — the radio and the 64 KiB RAM feel it. Lesson 2 tested all three sets on the board.

1. Why Post-Quantum (3/8) ML-KEM inside: key generation

- Built on **module learning with errors** (module-LWE): polynomials with 256 coefficients mod q = 3329, grouped into vectors of k polynomials.
- KeyGen draws 32 random bytes d and z; d expands into a seed ρ and noise seed σ.
  - A: a k × k matrix of polynomials, expanded from ρ with SHAKE128 — never sent
  - s, e: small secret and noise polynomials (coefficients in −η1 … η1), from σ
  - public **ek = (t = A·s + e, ρ)**; secret **dk = (s, ek, H(ek), z)**
- Recovering s from t is the hard problem: no efficient classical or quantum algorithm is known.

Parameter set	**k**	**η1**	**d_u, d_v**	**ek = 384·k + 32**	**ct = 32·(d_u·k + d_v)**	**Used here**
ML-KEM-512	2	3	10, 4	800 B	768 B	TBD1, TBD2
ML-KEM-768	3	2	10, 4	1,184 B	1,088 B	tested only (Lesson 2)
ML-KEM-1024	4	2	11, 5	1,568 B	1,568 B	TBD3

1. Why Post-Quantum (4/8) ML-KEM inside: encapsulation



- Encrypt computes $u = A^T \cdot r + e1$ and $v = t^T \cdot r + e2 + \lceil q/2 \rfloor \cdot m$ with fresh small noise, then compresses u to d_u and v to d_v bits per coefficient: that is the ciphertext size of (3/8).
- Anyone with ek can encapsulate; the result is random-looking. Only dk can turn ct back into m — and so into the same K.
- In AuthKEM the responder encapsulates to the initiator's fresh ek_eph (message_2); each side also encapsulates to the other's **long-term** ek from its credential (messages 3 and 4).

1. Why Post-Quantum (5/8) ML-KEM inside: decapsulation and implicit rejection

```
Simplified — FIPS 203, ML-KEM.Decaps
m'       = Decrypt(s, ct)                  # v - s^T u, rounded: m if ct is genuine
(K', r') = G(m' || H(ek))
ct'      = Encrypt(ek, m', r')              # re-encrypt with the same randomness
return  K'               if ct' == ct
        J(z || ct)       otherwise          # implicit rejection: a pseudo-random key
```

- Re-encrypting proves the ciphertext was built honestly (Fujisaki–Okamoto transform); a tampered ct gives a key unrelated to the sender's.
- No error is returned: the next MAC or AEAD tag simply fails. An attacker learns nothing from *how* decapsulation failed.
- Lesson 2's “reject KAT” checks exactly this pseudo-random key.

1. Why Post-Quantum (6/8) ML-KEM inside: what costs memory and time

Item	Size (ML-KEM-1024)	Where it hurts
one polynomial (256 × 16-bit)	512 B	every operation
matrix A (k × k = 16 polynomials)	8 KiB	KeyGen, Encrypt — and the re-encryption inside Decaps
secret key dk	3,168 B	kept by its owner
ciphertext, public key	1,568 B each	radio: 15 fragments each
decapsulation stack, reference code	34.1 kB	more than the free RAM above a TBD3 image
same with MLK_CONFIG_LOW_RAM (one row of A at a time)	18.8 kB	fits: section 4

- Time is dominated by SHAKE128 (expanding A) and the NTT, a number-theoretic FFT that multiplies polynomials with 16-bit arithmetic mod 3329 — far cheaper on this core than the 256-bit field arithmetic of P-256.
- On the board, one TBD3 decapsulation step (I: m2) takes ≈ 83 ms (section 5).

1. Why Post-Quantum (7/8) Why not just swap in a new suite?

- Methods 1–3 need a **static Diffie-Hellman** key. No standard post-quantum algorithm offers static DH, so they cannot be made post-quantum.
- Method 0 with ML-DSA signatures works (pqsuites draft), but ML-DSA signatures are 2.4–4.6 KB and slow to verify on a small core.
- **AuthKEM** (draft-ietf-lake-authkem-edhoc, method 5): each side proves its identity by **decapsulating** a ciphertext made for its long-term ML-KEM key.
- Only KEM operations, AEAD and hashing: no signature, no DH, no shared symmetric key (PSK) — but trusted credentials and private keys, as always (box below).

"Does post-quantum EDHOC need a PSK?" → This AuthKEM implementation needs **no shared symmetric PSK**. It still needs a long-term ML-KEM private key on each side and a **trusted binding** of each peer's identity to its public key (its credential in the trust anchors).

1. Why Post-Quantum (8/8) Standards status

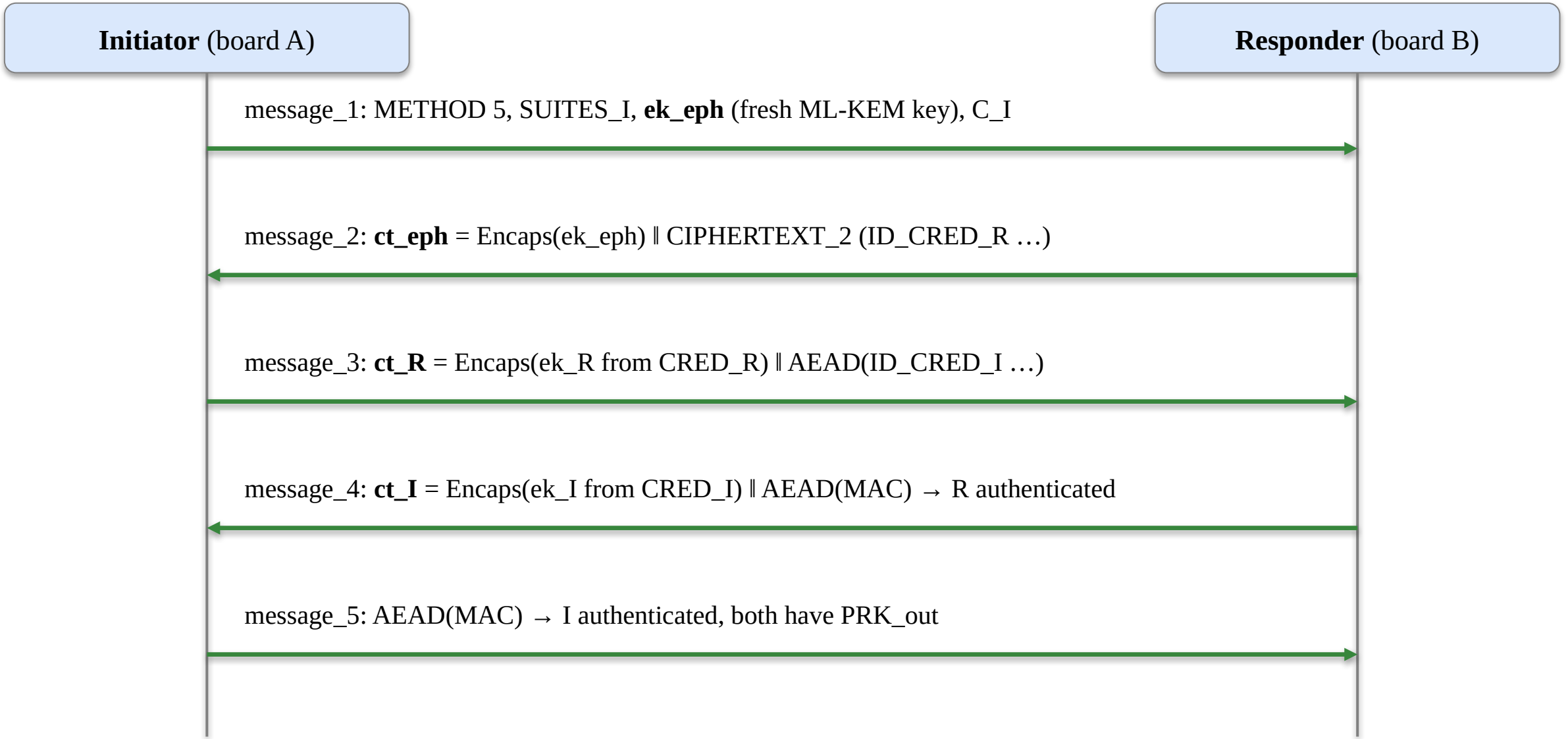
Document	Status (2026-10)	Defines
draft-ietf-lake-authkem-edhoc-01	IETF working-group draft	method 5: the 5-message protocol
draft-ietf-lake-pqsuites-01	IETF working-group draft	cipher suites TBD1, TBD2, TBD3
draft-ietf-jose-pqc-kem-06	IETF working-group draft	COSE key format for ML-KEM (AKP)
FIPS 203	NIST standard (2024)	ML-KEM itself

- “TBD” = the IANA numbers are not assigned yet. Until then the suites travel as RFC 9528 **private-use** labels: TBD1 = −22, TBD2 = −24, TBD3 = −23.
- The ML-KEM COSE algorithm values are private-use too: −65537 (ML-KEM-512), −65539 (ML-KEM-1024).
- When IANA assigns them, the labels change in one place (`-DAUTHKEM_SUITE_TBD2=<n>` …). A **new draft revision** is different: it can change messages or the key schedule and needs a review against the new text, the reference model and its test vectors.

Outline

1. Why Post-Quantum
2. **The AuthKEM Protocol**
3. Post-Quantum Suites
4. Fitting in 64 KiB
5. What to Expect
6. Practice

2. The AuthKEM Protocol (1/9) Five messages



2. The AuthKEM Protocol (2/9) What a KEM gives — and what it does not

Question	Answer
What does a KEM produce?	a ciphertext ct for one public key, and a shared secret ss that only the holder of the matching private key can recover from ct
Does encapsulating to a public key prove who I talk to?	**No.** It only proves: whoever answers correctly holds the private key of *that* public key. An attacker can send you their own public key.
So what makes it authentication?	(1) the public key comes from a credential you **trust** (trust anchors), and (2) the peer **proves possession** of the private key: its MAC is keyed with ss

- The same two parts exist in classical EDHOC: a trusted CRED, plus a signature or static-DH MAC that only the key owner can make.
- In AuthKEM the trust anchors are *cred_array* (the peer's CCS, found by its kid); the proof is MAC_2 (message_4) and MAC_3 (message_5).

2. The AuthKEM Protocol (3/9) How a KEM proves who you are

- Each side has a long-term ML-KEM key pair; its public key *ek* is in its credential.
- The peer encapsulates to that *ek*: ciphertext + secret. **Only the owner of dk can decapsulate** and learn the same secret.
- The secret is mixed into the key schedule. The next MAC can only be correct if the sender decapsulated → proof of identity.

Secret	From	Mixed into	Proves
ss_eph	ek_eph / ct_eph (messages 1, 2)	PRK_2e	fresh session key (forward secrecy)
ss_R	ct_R to the responder's static key (message 3)	PRK_3e2m	responder, at message 4
ss_I	ct_I to the initiator's static key (message 4)	PRK_4e3m	initiator, at message 5

Transcript hashes TH_2 … TH_5 bind every ciphertext; PRK_out = EDHOC_KDF(PRK_4e3m, 7, TH_5).

2. The AuthKEM Protocol (4/9) Who is authenticated when

After	Initiator knows	Responder knows
message_2	a fresh shared secret ss_eph with *someone*	nothing about the initiator
message_3	—	ID_CRED_I (encrypted under K_3, so only the true R can read it); not yet proven
message_4	**R is authenticated**: MAC_2 is keyed by PRK_3e2m, which needs ss_R — only R's private key decapsulates ct_R	—
message_5	—	**I is authenticated**: MAC_3 is keyed by PRK_4e3m, which needs ss_I — only I's private key decapsulates ct_I

- Before message_4 the initiator must not trust anything it received; before message_5 the responder must not. The application starts only after EDHOC OK.

2. The AuthKEM Protocol (5/9) The key schedule, message by message

```
message_2 (R)  ss_eph, ct_eph = Encaps(ek_eph)
               TH_2         = H( H(message_1), ct_eph )
               PRK_2e       = Extract( TH_2, ss_eph )
               CIPHERTEXT_2 = PLAINTEXT_2 XOR EDHOC_KDF(PRK_2e, 0, TH_2, length)
message_3 (I)  ss_R, ct_R   = Encaps(ek_R)                        # ek_R from CRED_R
               TH_3         = H( TH_2, PLAINTEXT_2, CRED_R, ct_R )
               PRK_3e2m     = Extract( EDHOC_KDF(PRK_2e, 1, TH_3), ss_R )
               CIPHERTEXT_3 = AEAD( K_3, IV_3 ; PLAINTEXT_3 )       # labels 3, 4 on TH_3
message_4 (R)  ss_I, ct_I   = Encaps(ek_I)                        # ek_I from CRED_I
               TH_4         = H( TH_3, PLAINTEXT_3, CRED_I, ct_I )
               PRK_4e3m     = Extract( EDHOC_KDF(PRK_3e2m, 5, TH_4), ss_I )
               MAC_2        = EDHOC_KDF( PRK_3e2m, 2, context_2 with TH_4, 16 )
               CIPHERTEXT_4 = AEAD( K_4, IV_4 ; MAC_2 )             # labels 8, 9 on TH_4
message_5 (I)  TH_5         = H( TH_4, PLAINTEXT_4 )
               MAC_3        = EDHOC_KDF( PRK_4e3m, 6, context_3 with TH_5, 16 )
               CIPHERTEXT_5 = AEAD( K_5, IV_5 ; MAC_3 )             # labels 11, 12 on TH_5
both           PRK_out      = EDHOC_KDF( PRK_4e3m, 7, TH_5, hash_length )
```

- Only R can decapsulate ct_R, so only R knows PRK_3e2m and can make MAC_2; only I can decapsulate ct_I and read message_4. Each line is recomputed independently by *tools/authkem_ref.py*.

2. The AuthKEM Protocol (6/9) Compared with RFC 9528

	RFC 9528, methods 0–3	AuthKEM -01, method 5
Messages	3 (+ optional message_4)	5
Ephemeral exchange	DH: G_X, G_Y → G_XY	KEM: ek_eph → ct_eph → ss_eph
TH_2	H(G_Y, H(message_1))	H(H(message_1), ct_eph)
Static secret in PRK_3e2m	G_RX (static DH) or none (signature)	ss_R, decapsulated from ct_R
Responder proven by	MAC_2 or signature in message_2	MAC_2 in message_4
Initiator proven by	MAC_3 or signature in message_3	MAC_3 in message_5
Extra keys	–	K_4 / IV_4 (labels 8, 9), K_5 / IV_5 (11, 12)
PRK_out	EDHOC_KDF(PRK_4e3m, 7, TH_4)	EDHOC_KDF(PRK_4e3m, 7, TH_5)

- Why two more messages: R can only prove itself after I has encapsulated to R's key (message_3), and I only after R has encapsulated to I's key (message_4).

2. The AuthKEM Protocol (7/9) The messages on the wire (TBD3)

```
message_1, 1,574 bytes (edhoc-authkem/build-tbd3/loopback.txt)
05                                  METHOD = 5 (AuthKEM)
36                                  SUITES_I = -23: TBD3 as a private-use label
59 06 20  ... 1,568 bytes ...       ek_eph: ML-KEM-1024 public key
2D                                  C_I = -14
```

Message	Bytes	One byte string = ML-KEM part ‖ EDHOC part
message_2	1,573	59 06 22 · ct_eph (1,568) · CIPHERTEXT_2 (2: C_R, ID_CRED_R)
message_3	1,589	59 06 32 · ct_R (1,568) · CIPHERTEXT_3 (18: C_I, ID_CRED_I, 16-B tag)
message_4	1,604	59 06 41 · ct_I (1,568) · CIPHERTEXT_4 (33: MAC_2 as bstr 17, tag 16)
message_5	35	58 21 · CIPHERTEXT_5 (33: MAC_3 as bstr 17, tag 16)

- The ciphertext has a fixed length, so the receiver splits it off the front.
- Of 6,375 bytes, 6,272 (98 %) are ML-KEM keys and ciphertexts; EDHOC itself adds 103 bytes. TBD2 is the same with 800 / 768-byte keys: `05 37 59 03 20 …`.

2. The AuthKEM Protocol (8/9) Where the draft is unclear — and what we chose

Rule in draft -01	Two readings	Chosen	Implemented in	Model check
SALT_3e2m context	KDF table: TH_2 · text + Figure 3: TH_3	**TH_3**	authkem_msg3_gen / _process → authkem_prk_extract	message_3 AEAD tag
TH_3 order (§5.3.2 vs §5.3.3)	H(ct_R, TH_2, …) · H(TH_2, …, ct_R)	§5.3.3 + figure	authkem_th_calculate (th.c)	message_3 AEAD tag
PRK_out formula (§4.1.3)	no label, TH_4 · label 7, TH_5	**label 7, TH_5**	authkem_msg5_gen / _process	PRK_out vs firmware

- Implemented revision: draft-ietf-lake-authkem-edhoc-01. It has no published test vectors, so: an **independent model** (*tools/authkem_ref.py*, Python, written from the draft text) recomputes every hash, key, MAC and PRK_out from the bytes our firmware sent.
- Swapping in the other SALT_3e2m reading makes the model fail at the message_3 tag — the choice is checked, not assumed.
- Every decision is written down in *AUTHKEM/AUTHKEM_SPEC_DECISIONS.md*.

2. The AuthKEM Protocol (9/9) Credentials for ML-KEM keys

- Credential = CWT Claims Set (CCS) holding a COSE_Key of type **AKP** (draft-ietf-jose-pqc-kem-06) with the ML-KEM public key.
- Identified by key id: ID_CRED = `{4: h'01'}` (initiator), `{4: h'02'}` (responder), sent in compact form.
- Demo keys and credentials are generated at build time (*tools/gen_ccs.c*); `PROFILE=production` takes them from your key store.
- The application sees the peer's ID_CRED through the same `id_cred_check()` hook as Lesson 3; on the initiator it runs before message_3 is sent.

Outline

1. Why Post-Quantum
2. The AuthKEM Protocol
3. **Post-Quantum Suites**
4. Fitting in 64 KiB
5. What to Expect
6. Practice

3. Post-Quantum Suites (1/3) TBD1, TBD2, TBD3

SUITE=	EDHOC AEAD	Hash / KDF	KEM	App AEAD	Wire label
TBD1	AES-CCM-16-128-128	SHAKE256 / KMAC256	ML-KEM-512	AES-CCM-16-64-128	−22
TBD2 (default)	A256GCM	SHAKE256 / KMAC256	ML-KEM-512	A256GCM	−24
TBD3	A256GCM	SHA-384 / HKDF	ML-KEM-1024	A256GCM	−23

- The signature slot of each suite (ML-DSA-44 / ML-DSA-87) is only for X.509 credentials; with CCS credentials no ML-DSA code is linked.
- One suite per image: `make … SUITE=TBD3` → *build-tbd3/*.

3. Post-Quantum Suites (2/3) The strongest configured option: TBD3

TBD3 = AES-256-GCM + ML-KEM-1024 + SHA-384

- The strongest option **configured in this tutorial**: the largest ML-KEM parameter set (category 5, ≈ AES-256), AES-256-GCM and SHA-384 — the same algorithm choices as NSA's CNSA 2.0 list.
- Assumptions: ML-KEM's security as standardised in FIPS 203, a trusted credential per peer, and good randomness (the demo profile does not have it: Lesson 3).
- Costs, all measured later: 1.6 KB messages = 15 radio fragments each, the largest stack reserve (23,808 B), and ≈ 3× the compute of TBD2.
- Answers the question “is there an option combining AES-256 and ML-KEM-1024?” — yes, and it runs on the board (section 5).

3. Post-Quantum Suites (3/3) Where each primitive comes from

Primitive	Implementation	Suites
ML-KEM-512 / 1024	mlkem-native (FIPS 203), with a local low-RAM option	TBD1–2 / TBD3
SHAKE256, KMAC256	common/crypto/keccak_kmac.c	TBD1, TBD2
SHA-384, HMAC/HKDF	common/crypto	TBD3
AES-CCM, AES-GCM	common/crypto (T-table AES)	all

- Same shared files as Lessons 2 and 3; the crypto tests of Lesson 2 cover them.
- SHAKE256 / KMAC256 and A256GCM were also checked against Python's hashlib, pycryptodome and *cryptography*.

Outline

1. Why Post-Quantum
2. The AuthKEM Protocol
3. Post-Quantum Suites
4. **Fitting in 64 KiB**
5. What to Expect
6. Practice

4. Fitting in 64 KiB (1/4) The stack problem

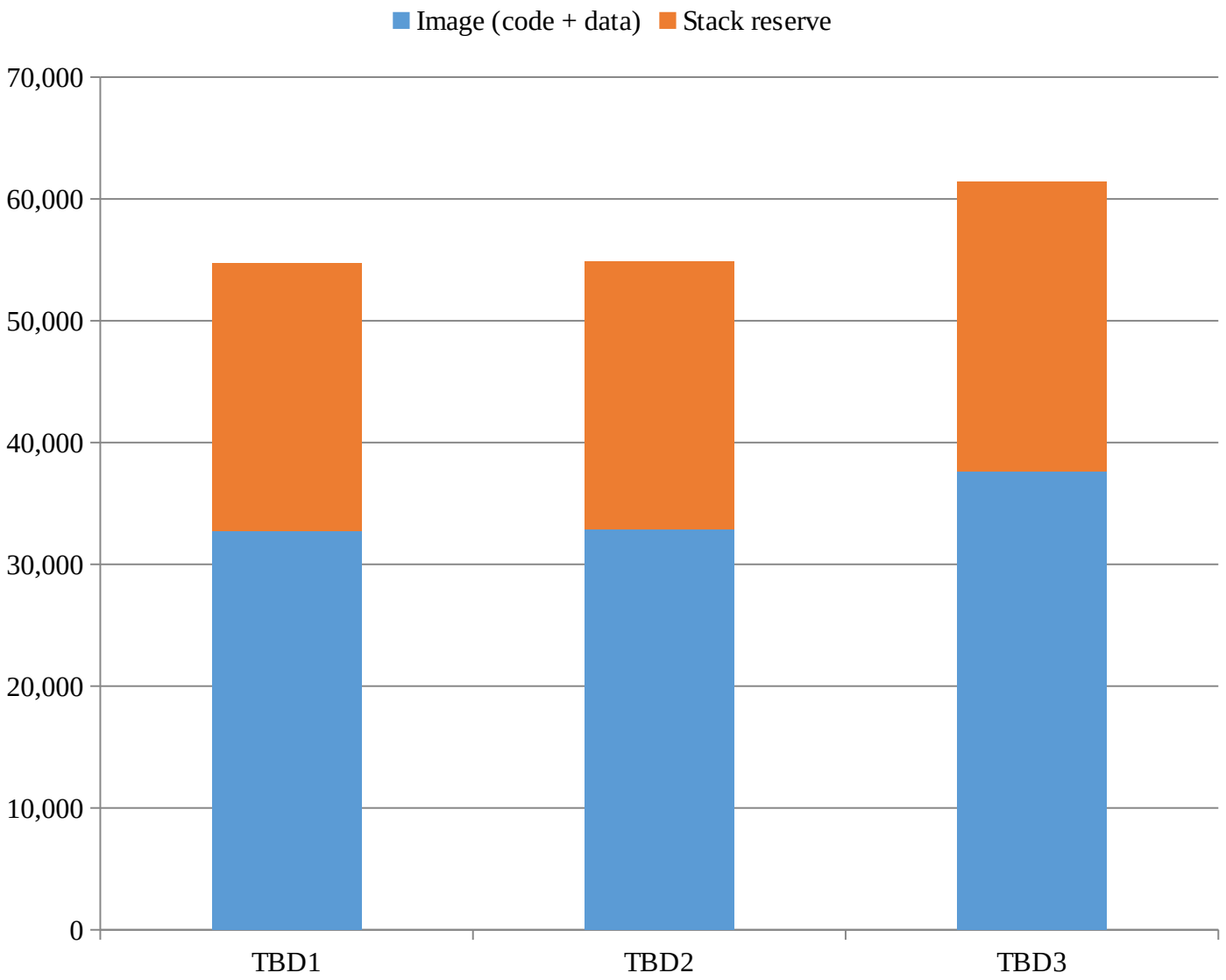
- ML-KEM-1024 decapsulation in the reference code needs **34.1 kB of stack**: it builds the whole 4 × 4 matrix of polynomials, then re-encrypts to check the ciphertext.
- Image (≈ 37.6 KB) + 34 kB stack > 64 KiB → TBD3 did not fit.
- Fix: a local option in our mlkem-native copy, `MLK_CONFIG_LOW_RAM`:
  - generate and use the matrix **one row at a time**
  - keep the decryption step in its own function, so its buffers are freed before re-encryption
- Result: **18.8 kB** for ML-KEM-1024 decapsulation. Outputs are bit-identical (the FIPS 203 known-answer tests of Lesson 2 still PASS).

4. Fitting in 64 KiB (2/4) Stack per role

Suite	Role	Peak on the board	Reserve (peak + 25 %)	Deepest call
TBD1 / TBD2	initiator	15,552 B	22,016 B	ML-KEM-512 decaps
TBD1 / TBD2	responder	13,680 B	18,688 B	
TBD3	initiator	18,592 B	23,808 B	ML-KEM-1024 decaps + re-encrypt
TBD3	responder	16,368 B	20,480 B	

- The responder decapsulates less deeply, so it gets a smaller reserve. That freed the room for the SD file transfer and `TIMING=1` in the same image (Lesson 5).
- Peaks measured with the stack paint of Lesson 2 during real handshakes.

4. Fitting in 64 KiB (3/4) Image sizes



- Largest build per suite (initiator), bytes.
- TBD3: 37,637 + 23,808 = **61,445 B of 65,536 B** (94 %).
- Less code than classical suite 25 (47,913 B): no elliptic-curve or signature library is linked.

4. Fitting in 64 KiB (4/4) Build and run

```
Run this — from fpga/sw/export/edhoc-authkem; last line = expected output
cd fpga/sw/export/edhoc-authkem
make edhoc_authkem_initiator edhoc_authkem_responder SUITE=TBD3 TIMING=1
../common/board/run_pair.sh akTBD3 build-tbd3/edhoc_authkem_initiator.elf \
                                   build-tbd3/edhoc_authkem_responder.elf
akTBD3 PASS hs=1.239387s stack_i=18592 stack_r=16368 prk=- 23s
```

```
Run this — on the PC; the last two lines = expected output
make unit-test SUITE=TBD2        # suite gate, KEM, CBOR codecs, error messages
make loopback-test SUITE=TBD3 LOOPBACK_LOG=build/loopback.txt
python3 tools/authkem_ref.py build/loopback.txt
  ok   PRK_out = EDHOC_KDF(PRK_4e3m, 7, TH_5, 48) equals the firmware's
REFERENCE MODEL: PASS
```

Board: same scripts as Lesson 3. PC: unit tests, loopback, and the independent model.

Outline

1. Why Post-Quantum
2. The AuthKEM Protocol
3. Post-Quantum Suites
4. Fitting in 64 KiB
5. **What to Expect**
6. Practice

5. What to Expect (1/5) Initiator console, TBD3

```
=== EDHOC Method 5 AUTHKEM (draft-ietf-lake-authkem-edhoc-01, pqsuites TBD3) Initiator (ML-KEM-1024) ===
PERF handshake_total   93591580 cyc 1.247887 s
PERF handshake_link    64279412 cyc 0.857058 s
PERF handshake_compute 29312168 cyc 0.390828 s
PERF msg1_build        1106262 cyc 0.014750 s
PERF msg2_process      6240840 cyc 0.083211 s
PERF msg3_build        7166204 cyc 0.095549 s
PERF msg4_process      9485599 cyc 0.126474 s
PERF msg5_build        2585856 cyc 0.034478 s
STACK peak 18592 B, reserve 23808 B, free 27408 B, headroom 8816
EDHOC OK!
APP: channel up (aead=3 key=32 iv=12 tag=16)
PERF encrypt      89688 cyc 0.001195 s 60 bytes 50209 B/s
APP DATA OK
```

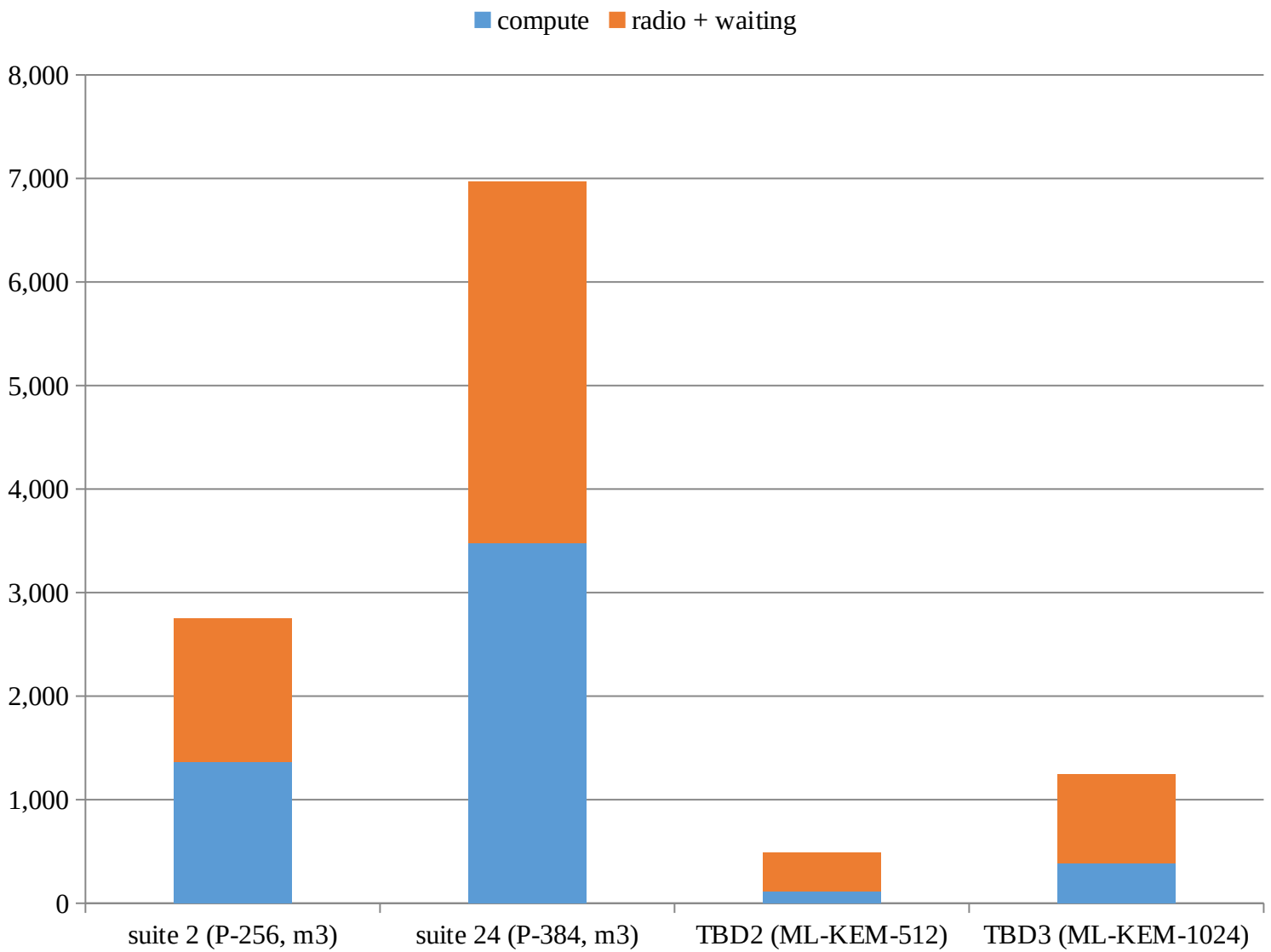
- *aead=3*: COSE algorithm 3 = A256GCM; 32-byte key. PRK_out is 48 bytes (SHA-384).
- Whole post-quantum handshake with ML-KEM-1024 + AES-256: **1.25 s**, of which only 0.39 s is computation.

5. What to Expect (2/5) Time of every message step (ms)

Suite	I: m1	R: m1 + m2	I: m2	I: m3	R: m3	I: m4	R: m4	I: m5	R: m5	I compute	Total
TBD1	1.2	25.9	32.8	27.0	35.1	36.3	28.2	3.2	3.2	116.7	502.6
TBD2	1.2	26.1	32.9	27.4	35.5	36.7	28.6	3.4	3.5	118.0	491.4
TBD3	14.8	83.1	83.2	95.5	105.4	126.5	109.4	34.5	32.2	390.8	1247.9

- I = initiator builds / processes, R = responder. The ephemeral ML-KEM key pair is made just before message_1 (counted in compute, not in a step).
- TBD3 is ~3× TBD2: ML-KEM-1024 does more work and SHA-384 / HKDF replaces KMAC.
- Every step is below 130 ms. The rest of the 0.5–1.25 s is the radio.

5. What to Expect (3/5) Post-quantum vs. classical



- Initiator, ms. **Post-quantum is faster** here: ML-KEM needs ~10× less computation than P-256 on this core.
- But messages are big: TBD2 sends 3.2 KB in total, TBD3 6.4 KB, vs. 103 B for the RFC 9529 P-256 example — so the radio share grows.

5. What to Expect (4/5) Message sizes (bytes)

	message_1	message_2	message_3	message_4	message_5	Total
RFC 9529 trace 2 (P-256, method 3)	39	45	19	–	–	103
RFC 9529 trace 1 (X25519, signatures)	37	116	90	–	–	243
TBD2 (ML-KEM-512)	806	773	789	804	35	3,207
TBD3 (ML-KEM-1024)	1,574	1,573	1,589	1,604	35	6,375

- Each of messages 1–4 carries one ML-KEM public key or ciphertext.
- A 6.4 KB handshake is ~60 radio frames: fragmentation and a reliable link (Lesson 5) matter more than raw speed.
- Sizes from the host loopback log (same encoder as the board).

5. What to Expect (5/5) What you should see

Run	You should see
TBD1 / TBD2 / TBD3 on two boards (run_pair.sh)	PASS; prk=match with DEBUG=1 builds
Host unit tests + loopback	PASS for each suite
Independent reference model (authkem_ref.py)	REFERENCE MODEL: PASS
Strongest configured option, TBD3 (AES-256 + ML-KEM-1024 + SHA-384)	1.25 s handshake, 61,445 B of 65,536 B
Shared symmetric PSK needed?	no — but trusted credentials and private keys are
Before real use	drafts can still change; code points are private-use labels until IANA assigns them

Outline

1. Why Post-Quantum
2. The AuthKEM Protocol
3. Post-Quantum Suites
4. Fitting in 64 KiB
5. What to Expect
6. **Practice**

6. Practice (1/3) Exercises

Exercise 1:

- Run TBD1, TBD2 and TBD3 on your boards with `TIMING=1`; fill in the step table.

Exercise 2:

- Run the reference model on a loopback log. Then change SALT_3e2m to the TH_2 reading in the model only: which check fails first, and why?

Exercise 3:

- Remove `-DMLK_CONFIG_LOW_RAM` from *mk/config.mk*, build TBD3 and run it. What do the consoles report? Why does the build still link? Restore the line afterwards.

6. Practice (2/3) What to put in the report

- Exercise 1: your table next to ours, and the ratio compute / total for each suite.
- Exercise 2: the model's output before and after your change; which draft sentence supports the reading we chose.
- Exercise 3: the STACK lines, and why a fixed reserve cannot catch this at link time.

6. Practice (3/3) Answer notes

Exercise	What you should see	Why
1. Step table	compute / total ≈ 23 % (TBD1), 24 % (TBD2), 31 % (TBD3)	the radio dominates; TBD3 computes ≈ 3× more
2. TH_2 for SALT_3e2m	the model fails at the message_3 AEAD tag; the checks before it still pass	K_3 / IV_3 come from PRK_3e2m, built from that salt
3. No LOW_RAM	decapsulation needs ≈ 34 kB, but only ≈ 27.7 kB is free above the 37.8 KB image: the stack runs into data — OVERRAN-RESERVE at best, a hang or FAIL at worst	the linker checks only the configured reserve (23,808 B), not the real demand

- Restore after exercise 3: `git checkout edhoc-authkem/mk/config.mk` (from *fpga/sw/export*), rebuild both roles, rerun: PASS.
- Negative cases to recognise: a peer whose credential is not in your trust anchors → EDHOC FAIL when its kid is looked up; a modified ciphertext → no error from decapsulation, but the next AEAD tag or MAC fails.

Today's lesson highlight:

- A quantum computer breaks elliptic-curve DH and signatures; ML-KEM (FIPS 203) replaces the key exchange.
- AuthKEM (method 5): 5 messages; a peer is authenticated by proving it can decapsulate to the public key in its trusted credential — no signature, no DH, no shared PSK.
- TBD3 = AES-256-GCM + ML-KEM-1024 + SHA-384 runs on the board: 1.25 s, fits 64 KiB thanks to a low-RAM ML-KEM and per-role stacks.
- Faster to compute than P-256, but 30–60× larger messages: the radio link becomes the bottleneck.

Next lesson:

「 Lesson 5 」 Secure file transfer and results

THANK YOU

[YYYY/MM]